Tutorial 1: LV Voltage Estimation
Exercise Solutions

Before you start

In a fresh kernel, run Sections 2.2–2.4 of `Tutorial_1_LV_Voltage_Estimation.ipynb`. Then run the seven listings below in order in the Exercise Workspace. Sections 2.5–2.6 are not needed. The listings use the notebook's existing data and helper functions.

Each fit uses 40 epochs, independently of quick/full mode. Select settings and a seed using three training-data folds; leave the test set until E2.3. This is same-time estimation from simulated data, not forecasting. The exercises reuse the tutorial's test period, not a new independent test set.

How to read this document. Each task is followed by its recorded output and a short explanation. Values come from the existing exercise CSV/JSON records and may vary slightly on another run. Selection uses unrounded scores, not the rounded values printed here. File-export code is omitted so the listings focus on the exercise itself.

E1.1: Start with 93 hidden neurons and tanh

Check this configuration on three blocked folds. `evaluate_config()` fits scalers on each fold's training rows and trains a fresh network.

```
e1_config = {
    "hidden_neurons": 93,
    "activation": "tanh",
    "lr": 1e-3,
    "weight_decay": 1e-5,
    "batch_size": 72,
    "epochs": 40,
}
e1_seed = 42
e1_k = 3
e1_rows = []
e1_scores = evaluate_config(
    e1_config, PQ_tr, V_tr, k=e1_k, seed=e1_seed
)
# Keep this result for the comparison in E1.3.
e1_row = e1_config.copy()
e1_row.update(e1_scores)
e1_rows.append(e1_row)
print("Mean RMSE (V):", round(e1_scores["RMSE_kfold"], 6))
print("Std. (V):", round(e1_scores["RMSE_std"], 6))
```

Recorded output: mean validation RMSE = 0.081666 V; standard deviation = 0.004465 V.

The mean summarises the three validation errors; the standard deviation shows how much they differ across folds, not a confidence interval.

E1.2: Increase the hidden-neuron count to 155

Copy the starting settings and change only the hidden-neuron count. Use the same seed and folds for a fair comparison.

```
e1_config_155 = e1_config.copy()
e1_config_155["hidden_neurons"] = 155
e1_scores_155 = evaluate_config(
    e1_config_155, PQ_tr, V_tr, k=e1_k, seed=e1_seed
)
e1_row_155 = e1_config_155.copy()
e1_row_155.update(e1_scores_155)
e1_rows.append(e1_row_155)
print("Mean RMSE (V):", round(e1_scores_155["RMSE_kfold"], 6))
print("Std. (V):", round(e1_scores_155["RMSE_std"], 6))
```

Recorded output: mean validation RMSE = 0.075256 V; standard deviation = 0.003544 V.

The larger tanh network has a lower validation RMSE in this comparison. E1.3 checks whether either ReLU network does better.

E1.3: Add ReLU and choose the best configuration

Evaluate the two ReLU networks, then combine them with the stored tanh results. Sort by mean validation RMSE so the first row is the best case.

```
for e1_neurons in [93, 155]:
    e1_relu_config = e1_config.copy()
    e1_relu_config["hidden_neurons"] = e1_neurons
    e1_relu_config["activation"] = "relu"
    e1_relu_scores = evaluate_config(
        e1_relu_config, PQ_tr, V_tr, k=e1_k, seed=e1_seed
    )
    e1_relu_row = e1_relu_config.copy()
    e1_relu_row.update(e1_relu_scores)
    e1_rows.append(e1_relu_row)

e1_results = pd.DataFrame(e1_rows).sort_values(
    "RMSE_kfold", kind="stable"
).reset_index(drop=True)
display(e1_results[[
    "hidden_neurons", "activation", "epochs",
    "RMSE_kfold", "RMSE_std",
]].round(6))

# Only size and activation changed; keep the other settings.
e1_best_row = e1_results.iloc[0]
e1_best_config = e1_config.copy()
e1_best_config["hidden_neurons"] = int(e1_best_row["hidden_neurons"])
e1_best_config["activation"] = e1_best_row["activation"]
print("Selected configuration:", e1_best_config)
```

Recorded output. Fold means and standard deviations are in volts.

Hidden neurons	Activation	Epochs	Mean RMSE (V)	Std. (V)
155	tanh	40	0.075256	0.003544
93	tanh	40	0.081666	0.004465
155	relu	40	0.088692	0.003253
93	relu	40	0.103469	0.008829

Select 155 hidden neurons with `tanh`. Learning rate 10^{-3}, L2 strength 10^{-5}, batch size 72 and 40 epochs remain unchanged.

E2.1: Choose a random seed

Compare seeds 0, 1 and 2 with the selected configuration and the same folds. The returned table is ranked by mean validation RMSE, lowest first. Take the seed from the first row.

```
e2_seed_results = run_seed_search(
    PQ_tr, V_tr, e1_best_config, seeds=[0, 1, 2], k=e1_k
)
display(e2_seed_results[[
    "seed", "RMSE_kfold", "RMSE_std"
]].round(6))
e2_selected_seed = int(e2_seed_results.iloc[0]["seed"])
print("Configuration:", e1_best_config)
print("Selected seed:", e2_selected_seed)
```

Recorded output

Seed	Mean validation RMSE (V)	Std. (V)
1	0.069666	0.003863
0	0.075048	0.002759
2	0.076155	0.002292

Selected seed: 1.

This seed has the lowest mean validation RMSE here. Keep it and the selected settings for the final training step; the test set is still unused.

E2.2: Train a fresh model on all training rows

The settings are now fixed. Fit new scalers on all supplied training rows, then train a new network. Do not continue training a validation-fold model.

```
e2_input_scaler = MinMaxScaler().fit(PQ_tr)
e2_output_scaler = MinMaxScaler().fit(V_tr)
e2_X_train = e2_input_scaler.transform(PQ_tr).astype(np.float32)
e2_Y_train = e2_output_scaler.transform(V_tr).astype(np.float32)

e2_model = build_model(
    2 * C, C, e1_best_config, seed=e2_selected_seed
)
e2_history = train_model(
    e2_model, e2_X_train, e2_Y_train, e1_best_config
)
e2_model.summary()
print("Completed epochs:", len(e2_history.epoch))
```

Recorded output. The network part of the summary is:

Layer	Output width	Network parameters
Hidden Dense (tanh)	155	9,765
Output Dense (linear)	31	4,836
Total trainable network parameters		14,601

The model has 62 inputs and 31 outputs. It completed 40 epochs on all 6,048 training rows. Keras may additionally list optimizer-state parameters; these are not extra trainable network weights.

The fold models helped select settings. This new model is the one used to estimate the test voltages.

E2.3: Predict test voltages and calculate errors

Transform the test inputs with the fitted training scaler. `predict_unscaled()` converts predictions back to volts before the errors are calculated. Do not fit either scaler on the test set.

```
e2_X_test = e2_input_scaler.transform(PQ_te).astype(np.float32)
e2_V_estimated = predict_unscaled(
    e2_model, e2_X_test, e2_output_scaler
)
e2_error = e2_V_estimated - V_te

# Overall errors: average over all times and customers.
e2_rmse = np.sqrt(np.mean(e2_error ** 2))
e2_mae = np.mean(np.abs(e2_error))
e2_metrics = pd.DataFrame({
    "Metric": ["RMSE", "MAE"],
    "Value (V)": [e2_rmse, e2_mae],
})

# One RMSE per customer: average over time only (axis=0).
e2_customer_rmse = np.sqrt(np.mean(e2_error ** 2, axis=0))
e2_customer_errors = pd.DataFrame({
    "Customer": np.arange(1, C + 1),
    "RMSE (V)": e2_customer_rmse,
})
display(e2_metrics.round(6))
display(e2_customer_errors.round(6))
```

Recorded output. Evaluation uses 2,016 test rows and all 31 customers.

Metric	Value (V)
RMSE	0.054894
MAE	0.039941

RMSE gives more weight to larger errors; MAE is the mean absolute error. The customer table below shows whether the overall score hides customers with larger errors.

E2.3: Customer-level output

These are the per-customer errors from the same test predictions above. Customer 18 has the largest RMSE; E2.4 inspects its voltage trace.

Table 1: E2.3 output: test RMSE for every customer.

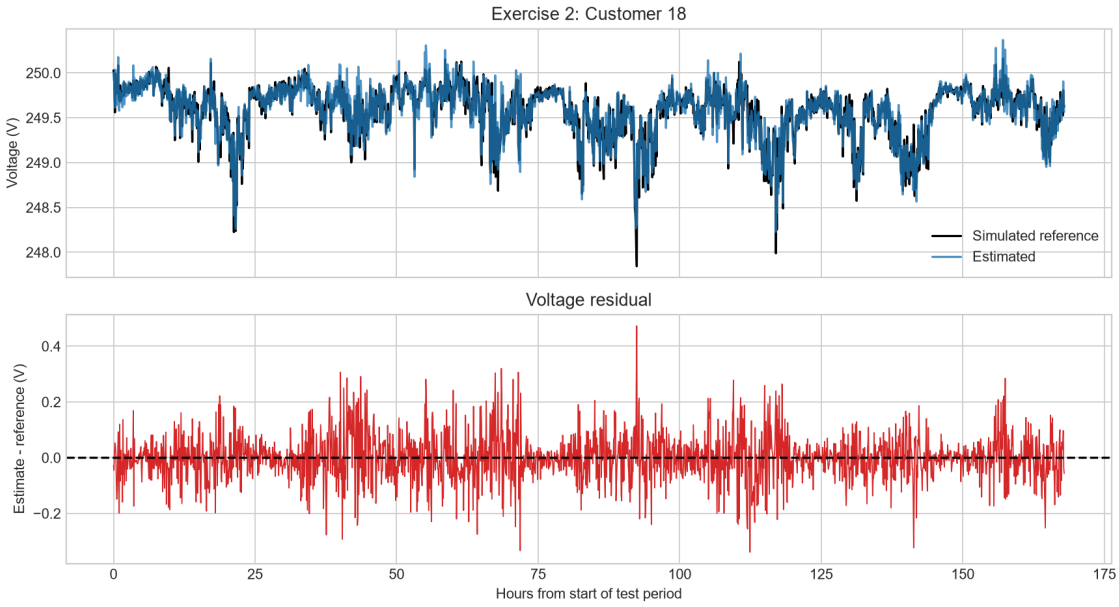
Customer	RMSE (V)
1	0.038563
2	0.039443
3	0.041992
4	0.041317
5	0.040860
6	0.038897
7	0.045386
8	0.041216
9	0.051875
10	0.046413
11	0.052481
12	0.048988
13	0.069180
14	0.032063
15	0.040150
16	0.045766
17	0.077979
18	0.085399
19	0.073828
20	0.054317
21	0.076296
22	0.059683
23	0.067354
24	0.039922
25	0.053868
26	0.075472
27	0.055716
28	0.042674
29	0.058709
30	0.048059
31	0.062222

E2.4: Inspect the customer with the largest RMSE

Use `argmax()` to find the largest customer error. Array positions start at zero, so add one for the customer number. Plot its voltage and residual (estimate minus reference); do not use this inspection to retune.

```
e2_worst = int(np.argmax(e2_customer_rmse))
print("Customer:", e2_worst + 1)
print("RMSE (V):", round(e2_customer_rmse[e2_worst], 6))
e2_hours = np.arange(len(V_te)) * MINUTES_PER_STEP / 60
e2_fig, e2_axes = plt.subplots(2, 1, figsize=(11, 6), sharex=True)
e2_axes[0].plot(
    e2_hours, V_te[:, e2_worst],
    color="black", label="Simulated reference"
)
e2_axes[0].plot(
    e2_hours, e2_V_estimated[:, e2_worst],
    color="tab:blue", label="Estimated", alpha=0.8
)
e2_axes[0].set_ylabel("Voltage (V)")
e2_axes[0].set_title(f"Exercise 2: Customer {e2_worst + 1}")
e2_axes[0].legend()
e2_axes[1].plot(
    e2_hours, e2_error[:, e2_worst],
    color="tab:red", linewidth=0.8
)
e2_axes[1].axhline(0, color="black", linestyle="--")
e2_axes[1].set_xlabel("Hours from start of test period")
e2_axes[1].set_ylabel("Estimate - reference (V)")
e2_axes[1].set_title("Voltage residual")
e2_fig.tight_layout()
plt.show()
```

Recorded output: customer 18; RMSE = 0.085399 V.



Reading the plot. The top panel compares the two voltage traces. The lower panel makes their small differences visible around the zero-error line. This customer has the largest RMSE in this run, not necessarily in every future dataset.